Implémentation des Widgets GTK4
Rapport Technique - Volume 2

Projet GTK Wrapper

8 mars 2026

Table des matières

1 Introduction **2**

2 Introduction **3**

3 Configuration de la Fenêtre (Header) **3**

4 Logique de Personnalisation Visuelle **3**

5 Construction de la Fenêtre **4**

1 Introduction

Ce rapport documente l'implémentation interne du wrapper GTK4. Contrairement au premier volume, centré sur les structures de configuration et l'usage de l'API, ce second document décrit la logique des fonctions, les appels GTK réellement utilisés et les choix techniques effectués dans chaque module.

Les extraits de code présentés ici sont basés sur l'implémentation réelle du projet. Les listings mettent l'accent sur les portions significatives (vérifications, branchements, gestion mémoire, raccordement des signaux) afin de conserver une lecture fluide dans le PDF.

L'objectif de ce volume est donc double : (1) expliquer *ce que fait* chaque fonction du wrapper et (2) expliciter *comment* elle le fait, notamment en termes de compatibilité GTK, d'ownership GLib et de comportement par défaut lorsque la configuration est absente.

[12pt,a4paper]report
[french]babel [T1]fontenc [utf8]inputenc listings xcolor [most]tcolorbox geometry
margin=2.5cm
winbox[1] enhanced, attach boxed title to top left=yshift=-2mm, xshift=2mm, colback=white, colframe=window-navy, fonttitle=**, colbacktitle=window-navy, title=1, boxed title style=size=small, sharp corners, drop shadow, bottom=2mm**
Gestion de la Fenêtre Principale (Windows)

2 Introduction

La fenêtre est le composant racine de toute application GTK. Ce module permet de définir les propriétés fondamentales comme le titre, la taille par défaut et l'apparence visuelle (fond coloré ou image). Il utilise les capacités de `GtkApplicationWindow` pour une intégration parfaite dans l'environnement de bureau.

3 Configuration de la Fenêtre (Header)

Structure : window_config La structure `window_config` regroupe toutes les caractéristiques physiques de la fenêtre. Elle permet de définir si la fenêtre est redimensionnable ou modale, ainsi que ses contraintes de taille minimale et maximale.

```c
typedef struct {
    const char *title;
    const char *icon_name;
    const char *background_color;
    const char *background_image_path;
    int default_width;
    int default_height;
    bool resizable;
    bool decorated;
    bool modal;
    widget_style_config style;
} window_config;
```

4 Logique de Personnalisation Visuelle

Fonction : apply_window_background Cette fonction utilise un `GtkCssProvider` pour injecter du CSS dynamiquement. Elle permet de définir une couleur de fond ou une image de fond avec la propriété `background-size: cover`, garantissant que l'image remplit toute la fenêtre.

```c
static void apply_window_background(const window_config *config, GtkWidget *window) {
    // ... Verification des parametres ...
    GtkCssProvider *provider = gtk_css_provider_new();
    char *css = g_strdup_printf(
        "window#%s { background-image: url('%s'); background-size: cover; }",
        name, config->background_image_path
    );
    gtk_css_provider_load_from_string(provider, css);
    // ... Application du style au display ...
}
```

5 Construction de la Fenêtre

Pour calculer la surface S disponible à l'écran en fonction des dimensions par défaut w et h, on utilise simplement :

$$S = w \times h$$

Fonction : create_window C'est la fonction principale du module. Elle instancie la fenêtre liée à l'application et applique toutes les configurations (titre, icône, taille, et style CSS) avant de la présenter à l'écran.

```
GtkWidget *create_window(GtkApplication *app, const window_config *config) {
    GtkWidget *window = gtk_application_window_new(app);
    if (config == NULL) return window;

    gtk_window_set_title(GTK_WINDOW(window), config->title);
    gtk_window_set_default_size(GTK_WINDOW(window), config->default_width, config->
        default_height);

    apply_window_background(config, window);
    apply_widget_style(window, &config->style);

    if (config->present_on_create) gtk_window_present(GTK_WINDOW(window));
    return window;
}
```